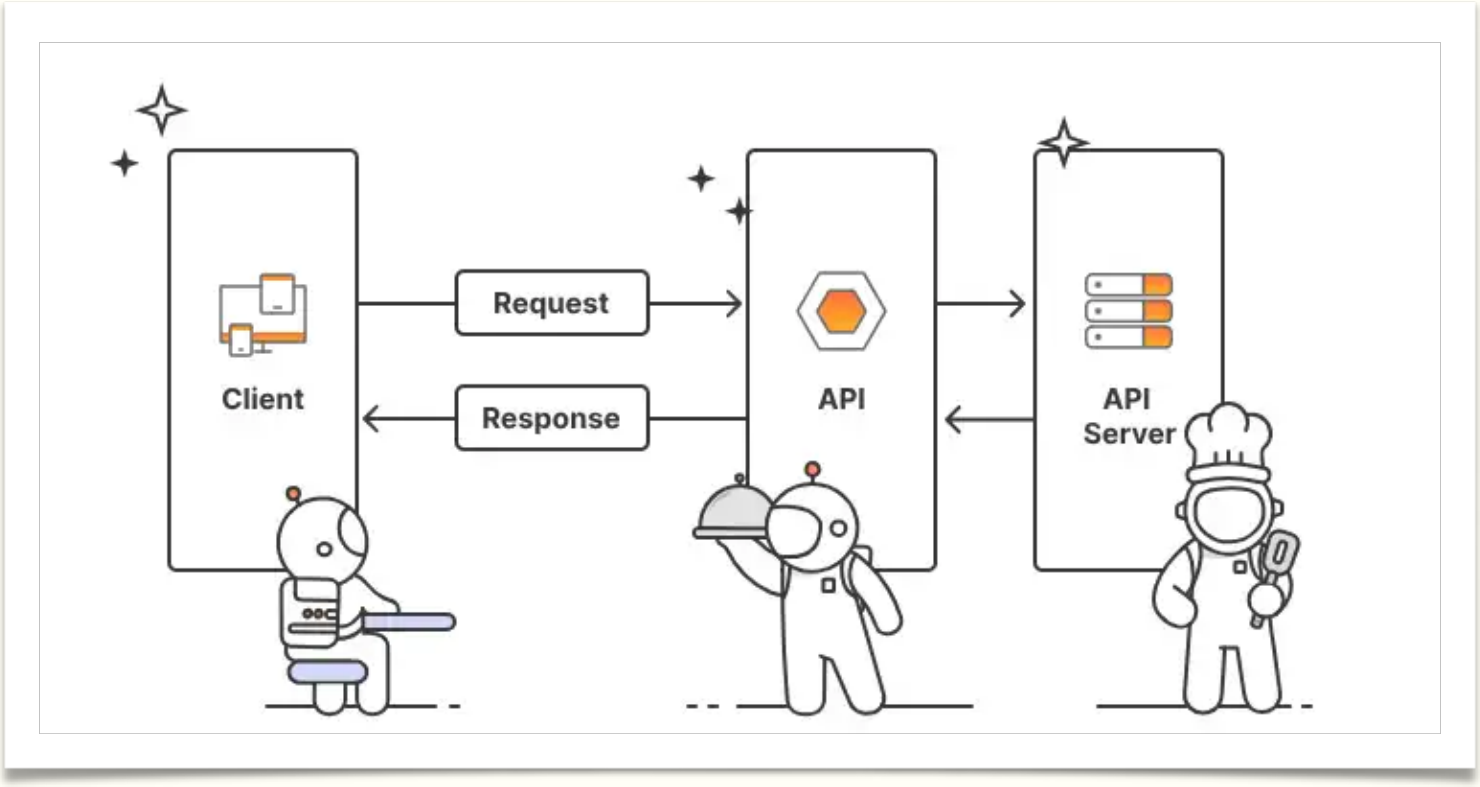
REST
Consumindo API’s



MENTES
MODO IA
EM AÇÃO

REST - Consumindo APIS externas

Vamos localizar API's* públicas, de preferência sem autenticação.



*API = Application Programming Interface

HACKA TRUCK Maker Space

Listas

```
[
Estudante(nome: "Hermione", idade: 11),
Estudante(nome: "Harry", idade: 10),
Estudante(nome: "Rony", idade: 12)
]
```



```
ForEach([ , , ] ){ aluno in

    Text(aluno.nome)

}
```

1ª Iteração - personagem = 

2ª Iteração - personagem = 

3ª Iteração - personagem = 

HACKA TRUCK Maker Space

Conceitos

JSON

Endpoint

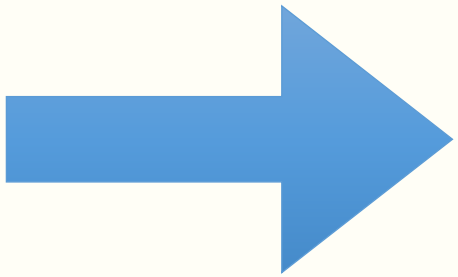
Objetos


```
[
  {
    "id": "9e3f7ce4-b9a7-4244-b709-dae5c1f1d4a8",
    "name": "Harry Potter",
    "alternate_names": [
      "The Boy Who Lived",
      "The Chosen One",
      "Undesirable No. 1",
      "Potty"
    ],
    "species": "human",
    "gender": "male",
    "house": "Gryffindor",
    "dateOfBirth": "31-07-1980",
    "yearOfBirth": 1980,
    "wizard": true,
    "ancestry": "half-blood",
    "eyeColour": "green",
    "hairColour": "black",
    "wand": {
      "wood": "holly",
      "core": "phoenix tail feather",
      "length": 11
    },
    "patronus": "stag",
    "hogwartsStudent": true,
    "hogwartsStaff": false,
    "actor": "Daniel Radcliffe",
    "alternate_actors": [],
    "alive": true,
    "image": "https://ik.imagekit.io/hpapi/harry.jpg"
  },
```

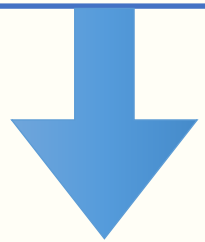

HACKA TRUCK Maker Space

Agora...

```
[
      {
      "nome" : "Hermione",
      "idade": 11
      },
      {
      “nome": "Harry",
       "idade": 10
      },
      {
      "nome": "Rony",
       "idade": 12
      }
]
```



```
[
Personagem(nome: "Hermione", idade: 11),
Personagem(nome: "Harry", idade: 10),
Personagem(nome: "Rony", idade: 12)
]
```



```
ForEach([ 👩 ,👦, 👦 ] ){ personagem in
    Text(personagem.nome)
}
```

HACKA TRUCK Maker Space

E no Swift?

Codable

Optional (?)

Objetos

```
struct HaPo: Codable, Identifiable {
    let id: String
    let name: String?
    let alternate_names: [String]?
    let species: String?
    let gender: String?
    let house: String?
    let dateOfBirth: String?
    let yearOfBirth: Int?
    let wizard: Bool?
    let ancestry: String?
    let eyeColour: String?
    let hairColour: String?
    let wand: Wand
    let patronus: String?
    let hogwartsStudent: Bool?
    let hogwartsStaff: Bool?
    let actor: String?
    let alternate_actors: [String]?
    let alive: Bool?
    let image: String?
}

struct Wand: Codable {
    let wood: String?
    let core: String?
    let length: Double?
}
```

Model

HACKA TRUCK Maker Space

Optional (?)

Permite que a variável possa ser nula (mil) ou ter um valor.

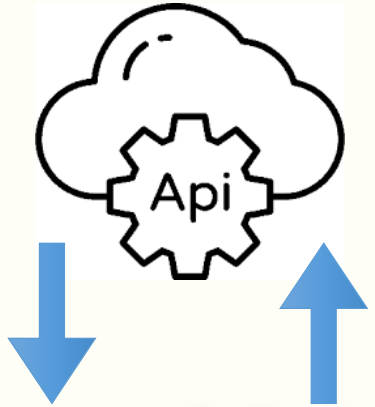
Para o Swift, String e String? são tipos diferentes.

Como transformar um tipo Optional para um tipo normal:

```
var opcional: String? = "não nulo"
guard let normal = opcional else { return }
print(normal) // não nulo
```

HACKA TRUCK Maker Space

Como Funciona o JSON

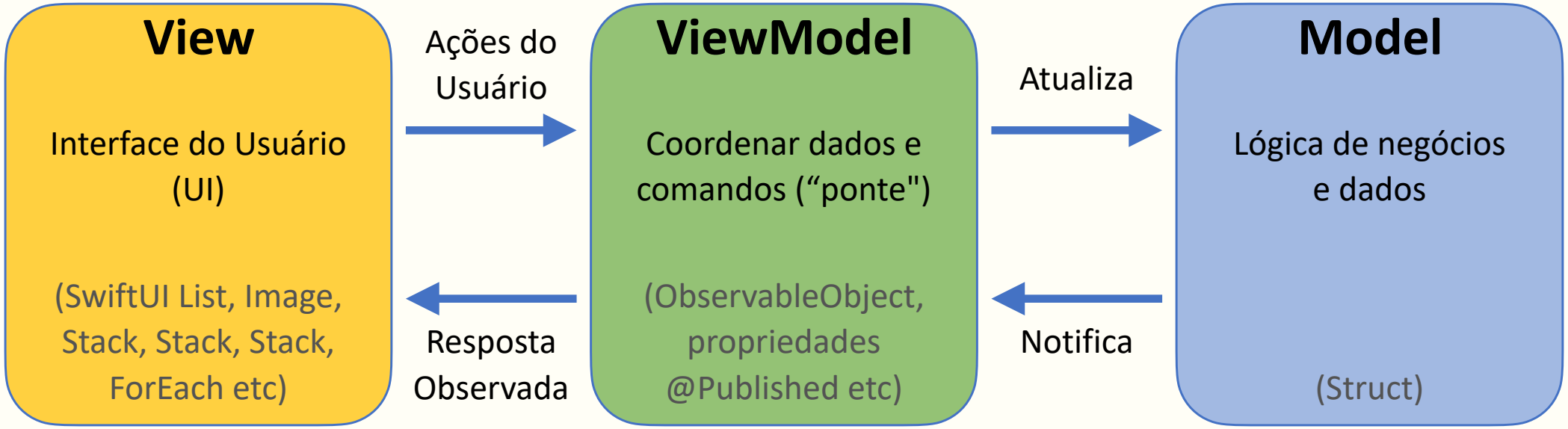


DECODIFICAR + ENCODIFICAR = CODIFICAR

Decodable + Encodable = Codable

HACKA TRUCK Maker Space

Swift - MVVM


View
Interface do Usuário (UI)
(SwiftUI List, Image, Stack, Stack, Stack, ForEach etc)
Ações do Usuário
Resposta Observada
ViewModel
Coordenar dados e comandos (“ponte")
(ObservableObject, propriedades @Published etc)
Atualiza
Notifica
Model
Lógica de negócios e dados
(Struct)

API Utilizada no Exemplo - HP-API

https://hp-api.onrender.com/api/characters/house/gryffindor

Use these routes to request data on Harry Potter characters:

View

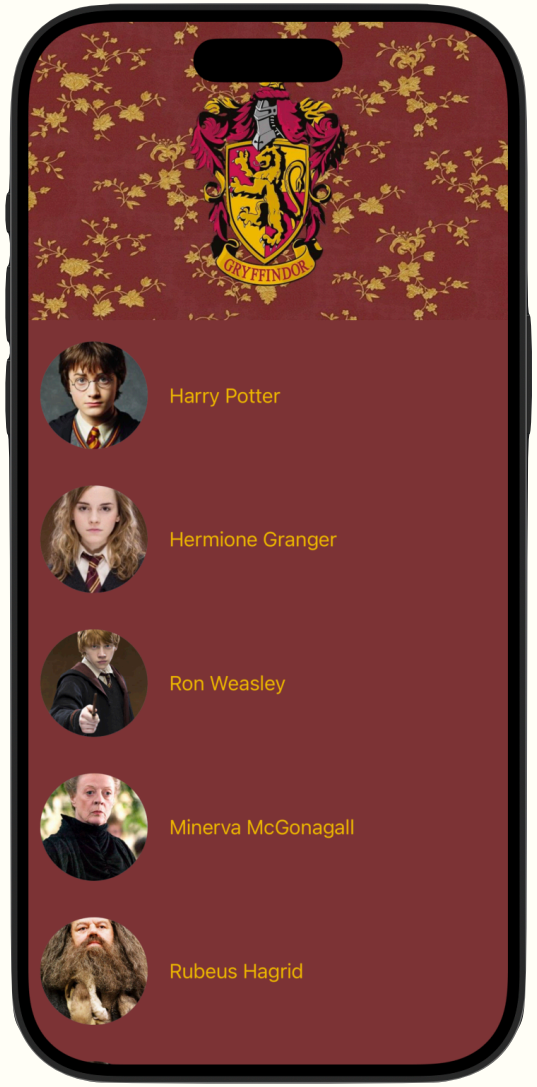
- all characters:
  - https://hp-api.onrender.com/api/characters
- a specific character, based on their unique ID:
  - https://hp-api.onrender.com/api/character/:id
- characters who are Hogwarts students during the book series:
  - https://hp-api.onrender.com/api/characters/students
- characters who are Hogwarts staff during the book series:
  - https://hp-api.onrender.com/api/characters/staff
- characters in a certain house, e.g. /gryffindor:
  - https://hp-api.onrender.com/api/characters/house/:house
- all spells:
  - https://hp-api.onrender.com/api/spells


```
[
  {
    "id": "9e3f7ce4-b9a7-4244-b709-dae5c1f1d4a8",
    "name": "Harry Potter",
    "alternate_names": [
      "The Boy Who Lived",
      "The Chosen One",
      "Undesirable No. 1",
      "Potty"
    ],
    "species": "human",
    "gender": "male",
    "house": "Gryffindor",
    "dateOfBirth": "31-07-1980",
    "yearOfBirth": 1980,
    "wizard": true,
    "ancestry": "half-blood",
    "eyeColour": "green",
    "hairColour": "black",
    "wand": {
      "wood": "holly",
      "core": "phoenix tail feather",
      "length": 11
    },
    "patronus": "stag",
    "hogwartsStudent": true,
    "hogwartsStaff": false,
    "actor": "Daniel Radcliffe",
    "alternate_actors": [],
    "alive": true,
    "image": "https://ik.imagekit.io/hpapi/harry.jpg"
  },
```


HACKA TRUCK Maker Space

Desafio: Consumir a API de Harry Potter

[Link]





HACKA TRUCK Maker Space

Service - Swift File

```
struct Service {
    func fetchPersonagens(url: String) async throws -> [HaPo] {
       *guard let url = URL(string: url) else { return [] }
      **let (data, _) = try await URLSession.shared.data(from: url)
     ***let values = try JSONDecoder().decode([HaPo].self, from: data)
        return values
    }
}
```

* Transforma a nossa url de string para o tipo URL

** Busca os dados através da URL e nos retorna um json na variável ‘data'

*** Transforma os dados recebidos de JSON para um array de HaPo

HACKA TRUCK Maker Space

ViewModel - Swift File

```swift
@Observable*
class ViewModel {
    var personagens: [HaPo] = []

    private let service = Service()

    func fetch() async {
        let url = "https://hp-api.onrender.com/api/characters/house/gryffindor"**
        do {
            personagens = try await service.fetchPersonagens(url: url)
        } catch {
            print(error.localizedDescription)
        }
    }
}
```

* Permite que a ViewModel possa ser um State na View

** URL que vamos receber os dados desejados (endpoint)

HACKA TRUCK Maker Space

E como chamo a classe na View?



```
struct ContentView: View {

    @State private var viewModel = ViewModel()

    var body: some View {
```

O array para o ForEach será chamado por ‘viewModel.personagens’

Após a última chave dentro da *body*:

```
}
.task {
    await viewModel.fetch()
}
```

HACKA TRUCK Maker Space

/hackatruck-makerspace

@hackatruck_makerspace

@hackatruck_makerspace

www.hackatruck.com.br